JNEOPALLIUM · SELF-SUPERVISED MAINTENANCE GUARDIAN

Predict Failures Before You Have a Failure History

Label-free predictive maintenance that learns from your telemetry — and gets sharper from your operators, without a redeploy

On-premises · Shadow → Advisory · No labels required · Never actuates · BSD-3 core

01 You have the data. You don't have the labels.

Predictive maintenance usually asks for the one thing you don't have: a clean history of past failures.

- You have **years of telemetry** — pressure, flow, vibration, temperature, power.
- You have a maintenance log that is **patchy, free-text, or missing.**
- Classic supervised models need **labelled failures** you can't supply.
- So the project stalls before it starts — or ships a black box nobody trusts.
- There is a better starting point than 'wait until we have labels.'

02 Two ways to do predictive maintenance

Supervised (the usual ask)

- Needs a labelled failure history
- Blind to failure modes it never saw labelled
- Months of data collection before value
- Hard to start on a new asset

Self-supervised (this)

- Needs only ordinary telemetry
- Learns 'normal', flags any drift from it
- Value from day one, in shadow
- Cold-starts from fleet peers

03 The solution: learn 'normal', flag the drift

It predicts each sensor from the others. When reality drifts from what the neighbours imply, that gap is your early warning.

- **No labels.** Every training target is another sensor — so it trains on plain operating data.
- **Multi-sensor.** A real fault is a pattern across sensors, not one needle in the red.
- **Patient.** It flags only persistent, trending, consistent drift — not a one-second blip.
- **Calibrated to each asset** against its own healthy history, not a generic threshold.
- **Honest.** On an unfamiliar asset it says 'not sure' instead of guessing.

04 It names the problem — and the lead time

Not one anonymous 'anomaly score', but an actionable advisory.

- **Which** kind: bearing, cavitation, sensor fault, energy loss, oscillation — or an honest 'unknown'.
- **How urgent:** an estimated lead time from the degradation trend.
- **Why:** the sensors driving the call, so a human can act on it.
- **One advisory per developing fault**, not an alarm every second.
- Everything a planner needs to schedule an inspection — and nothing that touches the machine.

05 It gets smarter every shift — with no redeploy

The one form of supervision that arrives for free: your operators.

- Mark an advisory **confirmed** or **false-positive** — that's the whole interface.
- A false alarm **raises** that threshold; a real one keeps it keen — bounded and rate-limited.
- It learns **in place**: no rebuild, no restart, no downtime.
- It **freezes** learning during abnormal periods so a bad patch can't poison it.
- Learning is **persisted** — it survives a restart and can be rolled back.

06 Proof on the bench (be honest about what it means)

Deterministic tests on a realistic synthetic corpus — not a field accuracy claim.

0	5 / 5	14 + 5	ADVISORY
Failure labels used in training	Fault families separated without labels	Java + Python tests passing	Safety ceiling, never exceeded

07 What it will do — and what it won't claim

It will

- Flag developing degradation early
- Name the likely family + a lead time
- Cut its own false alarms from feedback
- Run entirely on your premises

It won't (yet / ever)

- Claim a field rate from a bench test
- Be certain of a family before confirmations
- Ever actuate, schedule, or trip a device
- Work without a mostly-healthy baseline

08 Safe by construction

The safety story is not a policy — it's in the code.

- **Advisory-only invariant** that cannot be switched off.
- **No path to an actuator** anywhere in the model.
- The **hard safety layer** (interlocks, gates) is deterministic and lives outside the learned path.
- **Domain-shift and uncertainty** are first-class, so it can decline to over-claim.
- **Open core (BSD-3)** — auditable end to end.

09 Deploys where your data already is

It meets your plant, not the other way around.

- Ingests **OPC UA** and **MQTT / Sparkplug B** — the tags you already publish.
- Runs **local**, or **HTTP / gRPC** across a fleet.
- Emits advisories to **JSONL, Kafka, or MQTT** — into your SIEM, historian, or dashboard.
- Starts in **shadow**, graduates to **advisory** — and never further.
- Trained from your historian in an afternoon; improving from feedback thereafter.

Send us a month of telemetry. We'll show you the drift.

No labels needed. No agent on the PLC. No actuation — ever.

Shadow-mode pilot on one asset class, on your premises, with your operators in the loop.

Start a label-free pilot